Functional Programming – Lambda Calculus

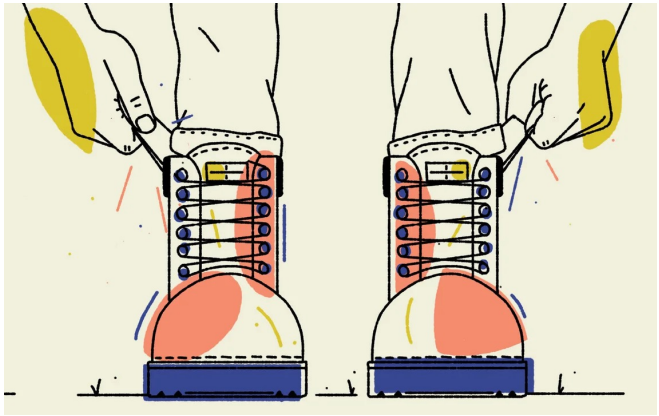
James Gain
Department of Computer Science
University of Cape Town
jgain@cs.uct.ac.za





What is the Lambda Calculus?

- Developed by Alonzo Church in the 1930's
- The 'machine code' basis of functional languages
- A very small precisely-defined programming language
  - Based on first-class anonymous mathematical functions
  - Easier to prove properties
  - But needs to be bootstrapped significantly to be practical



Turing vs. Church



Turing Machines

- Components:
  - An infinite tape, a read/write head, a state register, and transitional rules
- Operation:
  - Imperatively mutate state step by step





Lambda Calculus

- Components:
  - Variables, function abstraction, function application
- Operation:
  - Functionally rewrite expressions via reduction

- Both are universal theoretical models of computation and strictly equivalent
- By Church-Turing thesis both models can solve any real-world problem that has a step-by-step algorithm

Core Notation

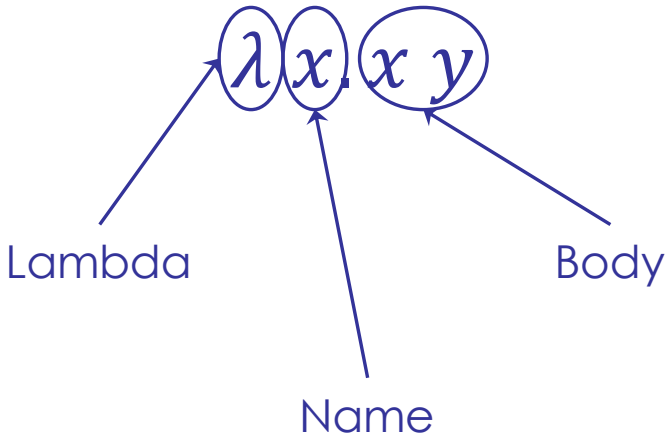
```
| lcletter     ⇐   a | b | c | ... | z
| name         ⇐   lcletter {lcletter}
| expression   ⇐   name | function | application
| function     ⇐   λ name . expression
| application  ⇐   expression expression
```

In EBNF notation ' | ' means OR, '{ }' means repetition

- Components:
  - Name (variable) – a collection of one or more letters
  - Function (abstraction) – defines a one-to-one mapping from an argument to an expression
  - Application – function invocation, calling a function (the 1st expression) with an argument (the 2nd expression)
  - Expressions – overarching concept that treats functions as first class

Function Components

- Lambda (λ)
  - Indicating an anonymous function follows
- Name
  - Single parameter of the function
  - Binds to identical names in the function body
- Body
  - The expression returned by the lambda when applied to the bound parameter
  - The '.' separates the function parameter from the body



Multi-argument Functions

- A list of multiple parameters is not supported in core Lambda Calculus
- Instead, nest functions with each successive function taking a different parameter
  - And use a shorthand notation
- This is called currying
- (Brackets not core but help with interpretation)

$$\lambda\, x\; y . x\; y$$

is shorthand for

$$\lambda\, x\, .\, (\lambda\, y . x\; y)$$

What is the curried version of:

$$\lambda\, x\; y\; z . y\; x\; z$$

Scope

- All names are local to function definitions
  - In $\lambda\, x\, .\, t$, the scope of x is t
- An occurrence of a variable x is *bound* if it occurs in the body t of a function $\lambda\, x\, .\, t$ where x is a parameter, otherwise it is *free*
- A closed expression has no free variables
- Examples: $(\lambda\, x.\, y\ x)(\lambda\, y.\, x\ y)$ $\qquad$ $\lambda\, x.\, (\lambda\, y.\, x\ y)$



Alpha Equivalence

- Sometimes the same names are bound to different scopes, which can lead to confusion
  - Fortunately, bound names are arbitrary and can be changed to make them unique
- Alpha equivalence
  - Two expressions are equivalent even if bound variables are renamed, so long as free variables are not changed
  - This change is call alpha conversion

$$\lambda\, x.(\lambda\, y.x\; y\; z)$$

$$=^{\alpha}$$

$$\lambda\, b.(\lambda\, a.b\; a\; z)$$

Quiz: Alpha Equivalence

- Which below is alpha equivalent to: $\lambda\, x\, y\, z.\, z\, x$

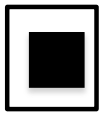
- $\lambda\, x.\, (\lambda\, y.\, (\lambda\, z.\, z))$
- $\lambda\, m\, n\, p.\, m\, n$
- $\lambda\, t\, o\, s.\, s\, t$
- $\lambda\, z\, y\, x.\, z\, x$

Quiz: Free or Bound?

- Which variables are bound (and to what parameters) and which are free?

$$\lambda\, x.\ \mathrm{y}\ (\lambda\, y.\ (\lambda\, \mathrm{x}.\ \mathrm{x}\ \mathrm{y}\ \mathrm{z}))$$

1 2 3 4 5 6 7



2 free, 7 free,
6 bound to 3,
5 bound to 4



2, 5, 6, 7 free



1 & 4 bound to 5,
2 & 3 bound to 6,
7 free



7 free,
5 bound to 1,
6 bound to 2

Application: β-reduction

- Recall `application ⇐ expression expression`
  - Application is the act of evaluating a function
  - Denoted by pairing two expressions: function and argument
- Carried out by β –reduction
  - Replace every free occurrence of the bound variable in the function body with the argument
  - Replaces a redex (reducible expression) with its evaluated form
  - Paradoxically, this may not actually reduce the length of the expression

$$(\lambda\, x.M)N \xrightarrow{\beta} M[x := N]$$

For example:

$$(\lambda\, x.x)y$$
$$\xrightarrow{\beta} x[x := y]$$
$$\rightarrow y$$

Beta Reduction Examples

1. Sep out function body
2. Substitute arg into free occurrences of parameter
3. Result is new expression

$$(\lambda x.x\ (\lambda x.x))(a\ b)$$
$$\xrightarrow{\beta}$$
$$(a\ b)\ (\lambda\ (a\ b).\ (a\ b))$$

Innermost x is not free in the enclosing scope

$$(\lambda x.x\ (\lambda x.x))(a\ b)$$
$$\xrightarrow{\beta}$$
$$(a\ b)\ (\lambda x.x)$$

$$(\lambda x.x\ x)(\lambda x.x\ x)$$
$$\xrightarrow{\beta}$$
$$(\lambda x.x\ x)(\lambda x.x\ x)$$
$$\xrightarrow{\beta}$$
$$(\lambda x.x\ x)(\lambda x.x\ x)$$
$$\xrightarrow{\beta}$$
$$\ldots$$

Normal Form

- β-normal form
  - Expression is in normal form when it contains no more redexes
  - i.e., β -reduction cannot be carried out any further
  - Equivalent to program termination
- Divergence
  - When a Lambda expression cannot be reduced to normal form
  - Equivalent to an infinite loop

Order of Reductions

Lazy

- Always choose leftmost / outermost redex
- Don't evaluate the argument if it can be avoided

$$(\lambda\, z.\, z\ a)\ ((\lambda\, x.\, x\ x)\ \lambda\, y.\, y)$$
$$\xrightarrow{\beta} ((\lambda\, x.\, x\ x))(\lambda\, y.\, y)\ a$$
$$\xrightarrow{\beta} (\lambda\, y.\, y\ \lambda\, y.\, y)\ a$$
$$\xrightarrow{\beta} (\lambda\, y.\, y)\ a$$
$$\xrightarrow{\beta} a$$

Strict

- Always choose rightmost / innermost redex
- Evaluate the argument before passing it to a function

$$(\lambda\, z.\, z\ a)\ ((\lambda\, x.\, x\ x)\ \lambda\, y.\, y)$$
$$\xrightarrow{\beta} (\lambda\, z.\, z\ a)\ (\lambda\, y.\, y\ \lambda\, y.\, y)$$
$$\xrightarrow{\beta} (\lambda\, z.\, z\ a)\ (\lambda\, y.\, y)$$
$$\xrightarrow{\beta} (\lambda\, y.\, y)\ a$$
$$\xrightarrow{\beta} a$$

Quiz: Reduction

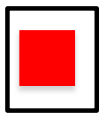
- What is the normal form of the Lambda expression reduced by lazy evaluation: $(\lambda\, x.\, \lambda\, y.\, x)\ (\lambda\, x.\, x)(\lambda\, z.\, z\ z)(\lambda\, z.\, z\ z)$



$(\lambda\, z.\, z\ z)(\lambda\, z.\, z\ z)$



$(\lambda\, x.\, x)$



$(\lambda\, z.\, z\ z)$



$(\lambda\, x.\, \lambda\, y.\, x)\ (\lambda\, x.\, x)(\lambda\, z.\, z\ z)(\lambda\, z.\, z\ z)$

- How about by strict evaluation?

Shorthand

- In the interests of sanity, Lambda function can be named using aliases
  - Not part of formal Lambda calculus, just a convenience
- Identity: $\mathrm{I} \equiv (\lambda\, \mathrm{x}.\, \mathrm{x})$
- Mockingbird (self-application): $\mathrm{M} \equiv (\lambda\, \mathrm{x}.\, \mathrm{x}\ \mathrm{x})$
- Discarding: $\mathrm{K} \equiv (\lambda\, \mathrm{x}.\, \lambda\, \mathrm{y}.\, \mathrm{x})$
- Infinite Loop: $\Omega \equiv ((\lambda\, \mathrm{z}.\, \mathrm{z}\ \mathrm{z})(\lambda\, \mathrm{z}.\, \mathrm{z}\ \mathrm{z}))$

Termination

- Reduce:

  $\mathrm{K\ I\ \Omega} \equiv (\lambda\,\mathrm{x}.\,\lambda\,\mathrm{y}.\,\mathrm{x})\ (\lambda\,\mathrm{x}.\,\mathrm{x})((\lambda\,\mathrm{z}.\,\mathrm{z\ z})(\lambda\,\mathrm{z}.\,\mathrm{z\ z}))$

Lazy

- $\mathrm{K\ I\ \Omega}$

  $\equiv (\lambda\,\mathrm{x}.\,\lambda\,\mathrm{y}.\,\mathrm{x})\ (\lambda\,\mathrm{x}.\,\mathrm{x})((\lambda\,\mathrm{z}.\,\mathrm{z\ z})(\lambda\,\mathrm{z}.\,\mathrm{z\ z}))$

  $\xrightarrow{\beta} (\lambda\,\mathrm{y}.\,\lambda\,\mathrm{x}.\,\mathrm{x})\ ((\lambda\,\mathrm{z}.\,\mathrm{z\ z})(\lambda\,\mathrm{z}.\,\mathrm{z\ z}))$

  $\xrightarrow{\beta} (\lambda\,\mathrm{x}.\,\mathrm{x})$

  $\equiv \mathrm{I}$
- Discards Ω

Strict

- $\mathrm{K\ I\ \Omega}$

  $\equiv (\lambda\,\mathrm{x}.\,\lambda\,\mathrm{y}.\,\mathrm{x})\ (\lambda\,\mathrm{x}.\,\mathrm{x})((\lambda\,\mathrm{z}.\,\mathrm{z\ z})(\lambda\,\mathrm{z}.\,\mathrm{z\ z}))$

  $\xrightarrow{\beta} (\lambda\,\mathrm{x}.\,\lambda\,\mathrm{y}.\,\mathrm{x})\ (\lambda\,\mathrm{x}.\,\mathrm{x})((\lambda\,\mathrm{z}.\,\mathrm{z\ z})(\lambda\,\mathrm{z}.\,\mathrm{z\ z}))$

  $\equiv KI\Omega$
- Repeats Ω infinitely

Lazy eval has properties described earlier (e.g., likelier termination).
In fact, this is where functional programming learnt it from.

Something Useful: Church Booleans

- Need to build computation – Boolean logic, arithmetic, loops, selection, etc.
- Boolean Types according to Church:
  - $\text{TRUE} \equiv (\lambda\, \text{x}.\lambda\, \text{y}.\text{x})$ (takes two inputs, returns the first)
  - $\text{FALSE} \equiv (\lambda\, \text{x}.\lambda\, \text{y}.\text{y})$ (takes two inputs, returns the second)
  - A Boolean is just a switch after all
- Conditionals:
  - $\text{IFTHEN} \equiv (\lambda\, \text{b}.\lambda\, \text{t}.\lambda\, \text{e}.\text{b t e})$
  - b is a Boolean type, t is the *then* branch, e is the *else* branch

Conditionals

- $IFTHEN\ \ TRUE$ a b

  $\equiv (\lambda\, \mathrm{b}.\lambda\, \mathrm{t}.\lambda\, \mathrm{e}.\, \mathrm{b\ t\ e})\ TRUE$ a b

  $\xrightarrow{\beta} (\lambda\, \mathrm{t}.\lambda\, \mathrm{e}.\, \mathrm{TRUE\ t\ e})$ a b

  $\xrightarrow{\beta} (\lambda\, \mathrm{e}.\, \mathrm{TRUE\ a\ e})$ b

  $\xrightarrow{\beta} TRUE$ a b

  $\equiv (\lambda\, \mathrm{x}.\lambda\, \mathrm{y}.\, \mathrm{x})$ a b

  $\xrightarrow{\beta x2}$ a

- $IFTHEN$ FALSE a b

  $\equiv (\lambda\, \mathrm{b}.\lambda\, \mathrm{t}.\lambda\, \mathrm{e}.\, \mathrm{b\ t\ e})$ FALSE a b

  $\xrightarrow{\beta} (\lambda\, \mathrm{t}.\lambda\, \mathrm{e}.\, \mathrm{FALSE\ t\ e})$ a b

  $\xrightarrow{\beta} (\lambda\, \mathrm{e}.\, \mathrm{FALSE\ a\ e})$ b

  $\xrightarrow{\beta}$ FALSE a b

  $\equiv (\lambda\, \mathrm{x}.\lambda\, \mathrm{y}.\, \mathrm{y})$ a b

  $\xrightarrow{\beta x2}$ b

Logical NOT

- Flips the state of a Boolean input
- $\text{NOT} \equiv (\lambda\, \text{b}.\, \text{b FALSE TRUE})$
- NOT TRUE

  $\equiv (\lambda\, \text{b}.\, \text{b FALSE TRUE})\ \text{TRUE}$

  $\xrightarrow{\beta} \text{TRUE FALSE TRUE}$

  $\xrightarrow{\beta} \text{FALSE}$

- NOT FALSE

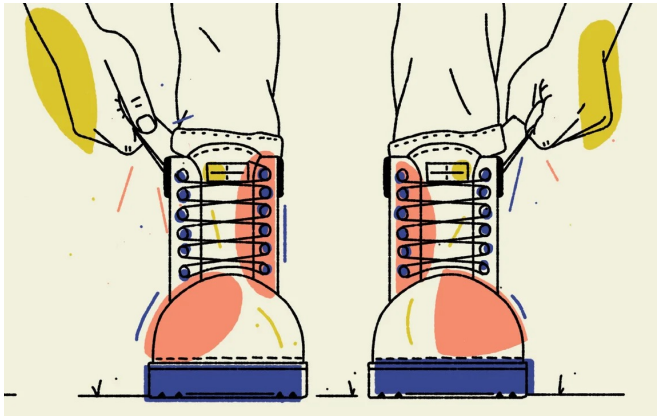
  $\equiv (\lambda\, \text{b}.\, \text{b FALSE TRUE})\ \text{FALSE}$

  $\xrightarrow{\beta} \text{FALSE FALSE TRUE}$

  $\xrightarrow{\beta} \text{TRUE}$

Bootstrapping

- Bootstrap other computation in a similar way to Church Booleans
- Church Numerals
  - A number N is a repetitive application of a function N times
- Loops through recursion
  - Using Church Booleans for termination
- Then just use the aliases in further computation



Summary

- Functional Programming
  - Functions rule everything
- Lazy evaluation
  - More likely to terminate and avoid unnecessary computation
- Recursion
  - How loops are realized in FP, use tail recursion to avoid stack overflow
- Lambda Calculus
  - The theoretical underpinnings of FP
- Exercises to come